

❑ Milestone 4 —Surveillance des ruptures en arrière-plan

Sprint V4 · Al Qalam Stock Manager Thème : threading · queue · Event · Lock · thread daemon

Objectif

Ajouter un **thread de surveillance** qui vérifie automatiquement les ruptures de stock toutes les N secondes, **sans bloquer l'interface graphique**, et affiche les alertes en temps réel dans un nouvel onglet.

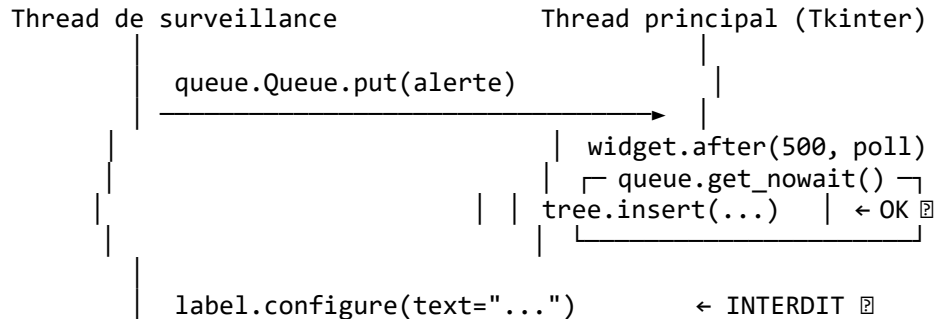
Ce que vous allez construire

```
alqalamv4/
├── services/
│   ├── stock_service.py      ← + threading.Lock sur les accès _produits
│   └── surveillance_service.py ← NOUVEAU : thread daemon + Queue + Event
├── ui/
│   ├── app.py                ← + onglet "⚠️ Alertes", start/stop thread
│   └── frames/
│       └── alertes_frame.py   ← NOUVEAU : polling Queue via after()
```

Concepts threading mis en œuvre

Concept	Classe / méthode	Rôle dans le projet
Thread daemon	threading.Thread(daemon=True)	Thread qui s'arrête automatiquement avec l'app
Arrêt propre	threading.Event · .set() · .wait(t)	Signal d'arrêt sans time.sleep
Exclusion mutuelle	threading.Lock · with self._lock:	Protège _produits contre les race conditions
Communication	queue.Queue · .put() · .get_nowait()	Thread → UI sans bloquer
Polling UI-safe	widget.after(500, callback)	Seul le thread principal modifie les widgets

Les règles d'or du threading Tkinter



Règle 1 : Ne jamais modifier un widget depuis un thread secondaire. **Règle 2** : Utiliser `queue.Queue` pour passer les données au thread principal. **Règle 3** : Utiliser `after()` côté UI pour lire la queue sans bloquer.

Guide de réalisation étape par étape

Étape 1 —Protéger StockService avec un Lock (15 min)

Dans `services/stock_service.py`, ajoutez un `threading.Lock` dans `__init__` et protégez toutes les méthodes qui lisent ou écrivent `_produits`.

Piège à éviter : Si `methode_A()` acquiert le Lock et appelle `methode_B()` qui acquiert aussi le Lock → **deadlock** !

DEADLOCK : Lock non réentrant

```
def entree_stock(self, ref, qte, note=""):
    with self._lock:
        produit = self.get_produit(ref)  # get_produit() essaie d'acquérir _lock → bl
        produit.qte += qte
```

CORRECT : accès direct sans appel imbriqué

```
def entree_stock(self, ref, qte, note=""):
    with self._lock:
        if ref not in self._produits:
            raise KeyError(f"Produit '{ref}' introuvable.")
        produit = self._produits[ref]  # accès direct, pas d'appel à get_produit()
        produit.qte += qte
```

Étape 2 —Créer SurveillanceService (20 min)

```
# services/surveillance_service.py
import threading, queue
```

```

from datetime import datetime

class SurveillanceService:
    def __init__(self, stock_service, intervalle=30):
        self._stock = stock_service
        self._intervalle = max(5, int(intervalle))
        self._stop_event = threading.Event() # signal d'arrêt
        self._lock = threading.Lock() # protège _historique
        self._file = queue.Queue() # canal vers l'UI
        self._historique = []
        self._thread = None

    def demarrer(self):
        self._stop_event.clear()
        self._thread = threading.Thread(
            target=self._boucle,
            name="SurveillanceStock",
            daemon=True, # s'arrête avec l'application
        )
        self._thread.start()

    def arreter(self):
        self._stop_event.set() # réveille le .wait() dans la boucle
        if self._thread and self._thread.is_alive():
            self._thread.join(timeout=5)

    def _boucle(self):
        while not self._stop_event.is_set():
            self._verifier()
            self._stop_event.wait(timeout=self._intervalle) # libère le CPU

    def _verifier(self):
        with self._lock:
            produits = self._stock.produits_en_alerte()
            if produits:
                alerte = {"timestamp": datetime.now().isoformat(), "produits": produits}
                self._file.put(alerte) # thread-safe
                with self._lock:
                    self._historique.append(alerte)

    def lire_alertes(self):
        """Vide la queue — appelé depuis l'UI via after()."""
        resultats = []
        try:
            while True:
                resultats.append(self._file.get_nowait())

```

```

except queue.Empty:
    pass
return resultats

@property
def est_actif(self):
    return self._thread is not None and self._thread.is_alive()

```

Étape 3 —Créer AlertesFrame (20 min)

```

# ui/frames/alertes_frame.py
import customtkinter as ctk
from tkinter import ttk

class AlertesFrame(ctk.CTkFrame):
    def __init__(self, parent, surveillance_service, **kwargs):
        super().__init__(parent, **kwargs)
        self._svc = surveillance_service
        self._id_poll = None
        self._construire_ui()
        self._demarrer_polling()

    def _demarrer_polling(self):
        # after() = planification sur le thread principal → UI-safe
        self._id_poll = self.after(500, self._poll)

    def _poll(self):
        alertes = self._svc.lire_alertes()      # Lecture non-bloquante
        for a in alertes:
            self._insérer(a)                    # modification widget OK ici
        self._id_poll = self.after(500, self._poll)  # replanifier

    def _insérer(self, alerte):
        # ... ajouter une ligne dans le Treeview ...
        pass

    def arreter_polling(self):
        """Appelé à la fermeture — évite un after() orphelin."""
        if self._id_poll:
            self.after_cancel(self._id_poll)

```

Étape 4 —Brancher dans app.py (10 min)

```
# Dans AlQalamApp.__init__
self.surveillance = SurveillanceService(self.stock, intervalle=20)
# ...construire onglets incluant AlertesFrame...
self.after(100, self._demarrer_surveillance) # après que mainloop soit active

def _demarrer_surveillance(self):
    self.surveillance.demarrer()

def _quitter(self):
    if messagebox.askyesno("Quitter", "Voulez-vous quitter ?"):
        self.alertes_frame.arreter_polling() # 1. stoppe after()
        self.surveillance.arreter()         # 2. signal + join()
        self.destroy()                     # 3. ferme la fenêtre
```

Critères de validation

- ☐ L'application démarre sans erreur et affiche 3 onglets
 - ☐ L'onglet "☐ Alertes" affiche le statut ☐ Thread actif
 - ☐ Les 4 produits en rupture apparaissent dans "Alertes actives" dès l'ouverture
 - ☐ Après une vérification automatique (≤ 20 s), une ligne s'ajoute dans l'historique
 - ☐ "☐ Vérifier maintenant" déclenche immédiatement une vérification
 - ☐ Une sortie stock qui crée une rupture est visible dans l'onglet Alertes
 - ☐ La fermeture de l'app est propre (pas de thread zombie)
-

Pour aller plus loin

- Ajouter un **son d'alerte** via winsound (Windows) ou playsound
 - Permettre de **régler l'intervalle** avec un slider directement dans l'UI
 - Utiliser threading.Timer à la place de Event.wait() pour la planification
 - Expérimenter avec concurrent.futures.ThreadPoolExecutor pour des vérifications parallèles
-